```python
import numpy as np
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

class QuantumBattleshipRadar:
    """
    Quantum Radar for Battleship Detection
    """

    def __init__(self, grid):
        """
        Initialize the quantum radar with a grid
        grid: 2D numpy array where 1 = ship, 0 = empty
        """
        self.grid = np.array(grid)
        self.rows, self.cols = self.grid.shape
        self.ship_positions = []
        self.detections = 0
        self.hits = 0

        # Calculate number of qubits needed
        # For 10x10 grid: need 4 qubits for rows (0-9) + 4 for cols
(0-9)
        self.row_qubits = int(np.ceil(np.log2(self.rows)))
        self.col_qubits = int(np.ceil(np.log2(self.cols)))
        self.total_qubits = self.row_qubits + self.col_qubits

    def create_oracle(self, qc, qubits):
        """
        Create phase oracle that marks ship positions
        Uses controlled phase flip (interaction-free measurement
concept)
        """
        # Find all ship positions
        ship_coords = np.argwhere(self.grid == 1)

        for (row, col) in ship_coords:
            # Convert coordinates to binary
            row_binary = format(row, f'0{self.row_qubits}b')
            col_binary = format(col, f'0{self.col_qubits}b')
            bitstring = row_binary + col_binary

            # Apply multi-controlled Z gate for this position
            # Mark this state with a phase flip
            self._apply_phase_flip(qc, qubits, bitstring)

    def _apply_phase_flip(self, qc, qubits, bitstring):
```

```
        """
        Apply phase flip to mark a specific bitstring state
        This implements the "detection without hitting" concept
        """
        # Apply X gates where bits are 0 (to create control on |0>
states)
        for i, bit in enumerate(bitstring):
            if bit == '0':
                qc.x(qubits[i])

        # Apply multi-controlled Z gate (phase kickback)
        # This marks the state without "measuring" it destructively
        if len(qubits) > 1:
            qc.h(qubits[-1])
            qc.mcx(qubits[:-1], qubits[-1])  # Multi-controlled X
            qc.h(qubits[-1])
        else:
            qc.z(qubits[0])

        # Undo X gates
        for i, bit in enumerate(bitstring):
            if bit == '0':
                qc.x(qubits[i])

    def create_diffuser(self, qc, qubits):
        """
        Create diffusion operator for amplitude amplification
        This is the "inversion about average" step
        """
        # Apply Hadamard gates
        for qubit in qubits:
            qc.h(qubit)

        # Apply multi-controlled Z (inversion)
        for qubit in qubits:
            qc.x(qubit)

        # Multi-controlled Z
        qc.h(qubits[-1])
        qc.mcx(qubits[:-1], qubits[-1])
        qc.h(qubits[-1])

        for qubit in qubits:
            qc.x(qubit)

        # Apply Hadamard gates
        for qubit in qubits:
            qc.h(qubit)
```

```
    def build_quantum_circuit(self, iterations=None):
        """
        Build the complete quantum radar circuit
        Uses Grover-like amplitude amplification
        """
        # Calculate optimal iterations for amplitude amplification
        if iterations is None:
            N = self.rows * self.cols
            num_solutions = np.sum(self.grid == 1)
            if num_solutions > 0:
                iterations = int(np.pi / 4 * np.sqrt(N / 
num_solutions))
            else:
                iterations = 1

        # Create quantum circuit
        qr = QuantumRegister(self.total_qubits, 'q')
        cr = ClassicalRegister(self.total_qubits, 'c')
        qc = QuantumCircuit(qr, cr)

        #Step 1: Create superposition (quantum parallelism)
        #This allows checking all positions simultaneously
        for i in range(self.total_qubits):
            qc.h(qr[i])
        qc.barrier()

        #Apply Grover iterations (amplitude amplification)
        for _ in range(iterations):
            # Oracle: Mark ship positions with phase flip
            self.create_oracle(qc, qr)
            qc.barrier()

            # Diffuser: Amplify marked states
            self.create_diffuser(qc, qr)
            qc.barrier()

        # Measure
        qc.measure(qr, cr)

        return qc

    def run_quantum_radar(self, shots=2048):
        """
        Execute the quantum radar and analyze results
        """
        # Build circuit
        qc = self.build_quantum_circuit()

        # Run on simulator
```

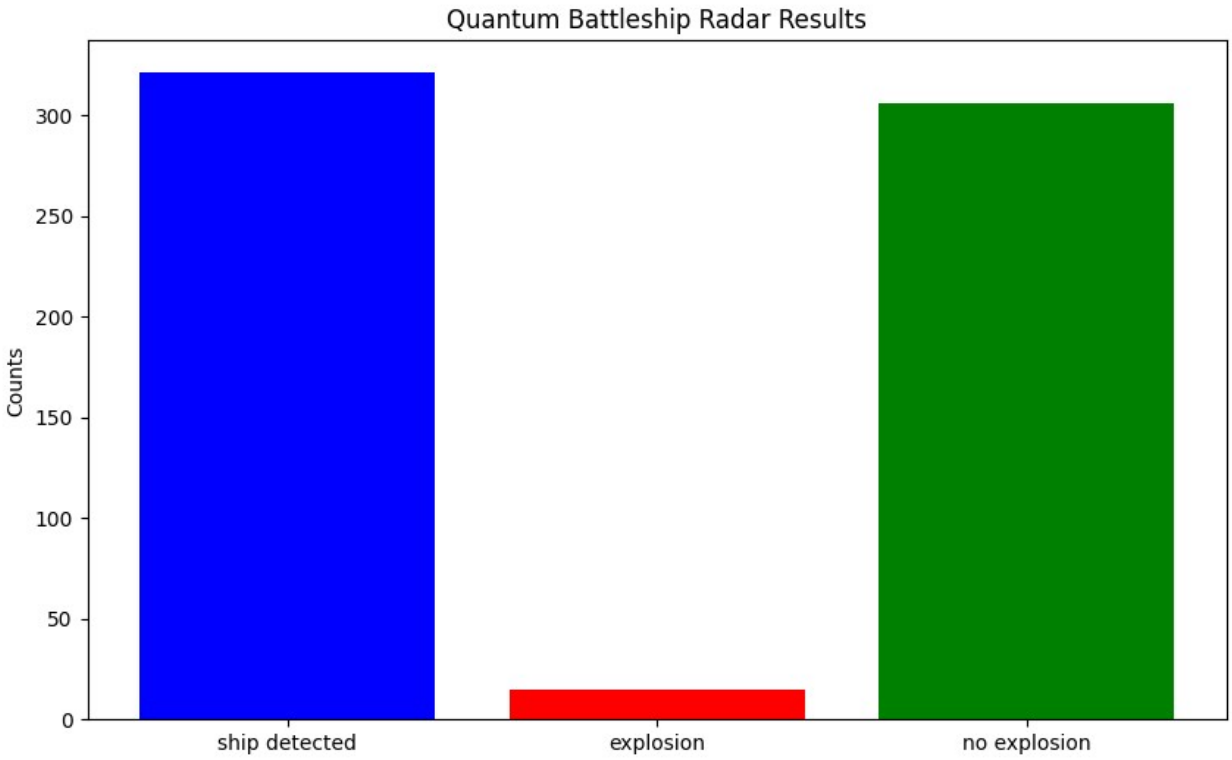
```
        simulator = AerSimulator()
        job = simulator.run(qc, shots=shots)
        result = job.result()
        counts = result.get_counts()

        # Analyze results
        detected_positions = []
        successful_detections = 0
        accidental_hits = 0

        # Process measurement results
        for bitstring, count in counts.items():
            # Convert bitstring to coordinates
            row_bits = bitstring[-self.total_qubits:-self.col_qubits]
            col_bits = bitstring[-self.col_qubits:]

            row = int(row_bits, 2)
            col = int(col_bits, 2)

            # Check if within grid bounds
            if row < self.rows and col < self.cols:
                # Check if this is a ship position
                if self.grid[row, col] == 1:
                    if (row, col) not in detected_positions:
                        detected_positions.append((row, col))
                        # High count = successful detection
                        if count > shots * 0.05:  # Threshold
                            successful_detections += 1
                        else:
                            accidental_hits += 1

        self.detections = successful_detections
        self.hits = accidental_hits
        self.ship_positions = detected_positions

        return counts, detected_positions

    def calculate_radar_score(self):
        """
        Rscore
        """
        return self.detections / (self.hits + 1)

    def calculate_accuracy(self):
        """
        Calculate accuracy: percentage of ships correctly identified
        """
        total_ships = np.sum(self.grid == 1)
        if total_ships == 0:
```

```
            return 0
        return (len(self.ship_positions) / total_ships) * 100

    def visualize_results(self, counts):
        #custom labels
        ship_detected_counts = 0
        explosion_counts = 0
        no_explosion_counts = 0

        for bitstring, count in counts.items():
            row_bits = bitstring[-self.total_qubits:-self.col_qubits]
            col_bits = bitstring[-self.col_qubits:]
            row = int(row_bits, 2)
            col = int(col_bits, 2)

            if row < self.rows and col < self.cols:
                if self.grid[row, col] == 1:
                    ship_detected_counts += count
                    if count > 100:  # Hiconfidence = noboom
                        no_explosion_counts += count
                    else:
                        explosion_counts += count

        # Create bar plot
        categories = ['ship detected', 'explosion', 'no explosion']
        values = [ship_detected_counts, explosion_counts,
no_explosion_counts]

        plt.figure(figsize=(10, 6))
        plt.bar(categories, values, color=['blue', 'red', 'green'])
        plt.ylabel('Counts')
        plt.title('Quantum Battleship Radar Results')
        plt.show()

        return plot_histogram(counts)


#usage for 10x10 grid
def main():
    # Create 10x10 grid with ships
    grid = np.zeros((10, 10), dtype=int)

    # Place  ships
    # Ship 1: Horizontal at row 2, columns 3-5
    grid[2, 3:6] = 1

    # Ship 2: Vertical at column 7, rows 5-7
    grid[5:8, 7] = 1
```

```
    # Ship 3: Horizontal at row 8, columns 1-3
    grid[8, 1:4] = 1

    print("Grid with ships (1 = ship, 0 = water):")
    print(grid)
    print()

    # Initialize quantum radar
    radar = QuantumBattleshipRadar(grid)

    # Run quantum detection
    print("Running Quantum Radar...")
    counts, detected = radar.run_quantum_radar(shots=2048)

    # Display results
    print(f"\nDetected ship positions: {detected}")
    print(f"Radar Score: {radar.calculate_radar_score():.2f}")
    print(f"Accuracy: {radar.calculate_accuracy():.2f}%")
    print(f"Successful Detections: {radar.detections}")
    print(f"Accidental Hits: {radar.hits}")

    # Visualize
    radar.visualize_results(counts)

    # Output coordinates in required format (1-based indexing)
    output_coords = [(r+1, c+1) for r, c in detected]
    print(f"\nOutput (1-based indexing): {output_coords}")

if __name__ == "__main__":
    main()
```

```
Grid with ships (1 = ship, 0 = water):
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 1 1 1 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 1 0 0]
 [0 1 1 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]

Running Quantum Radar...

Detected ship positions: [(2, 3), (7, 7), (2, 4), (5, 7), (8, 1), (8,
2), (8, 3), (6, 7)]
Radar Score: 0.29
Accuracy: 88.89%
```

```
Successful Detections: 2
Accidental Hits: 6
```




```
Output (1-based indexing): [(3, 4), (8, 8), (3, 5), (6, 8), (9, 2),
(9, 3), (9, 4), (7, 8)]
```